

Министерство цифрового развития

Федеральное государственное общеобразовательное учреждение высшего
образования

«Сибирский Государственный Университет Телекоммуникаций и
Информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Вариант 2.

Отчет по практической работе №4.

«Разработка и модульное тестирование класса Матрица на Java»

Работу выполнил студент группы ИП213:

Терновский Д. Р.
Соколовский Н. А.
Жуйков С. А.

Работу проверил ст. преподаватель кафедры ПМиК:

Агалаков А. А.

г. Новосибирск, 2025 г.

Цель работы

Сформировать практические навыки разработки на Java и модульного тестирования классов средствами IntelliJ IDEA/Android Studio.

Задание

Разработайте класс Матрица (Matrix) для операций матричной алгебры в соответствии с предложенной ниже спецификацией требований.

Разработайте тестовые наборы для тестирования методов класса на основе по критерию C2 (путей).

Выполните модульное тестирование класса средствами модульного тестирования Visual Studio.

Выполните анализ покрытия кода методов тестами.

Листинг программы

```
package com.example.matrixapplication;

public class Matrix {

    private final int rows;
    private final int columns;
    private final int[][] matrix;
    public Matrix(int[][] matrix, int rows, int columns) throws MatrixException {
        if (rows <= 0) {
            throw new MatrixException("Количество строк матрицы задано неверно");
        }
        if (columns <= 0) {
            throw new MatrixException("Количество столбцов матрицы задано неверно");
        }
        this.rows = rows;
        this.columns = columns;
        this.matrix = matrix;
    }

    public final Matrix sum(Matrix matrix) throws MatrixException {
        if (matrix.rows != this.rows || matrix.columns != this.columns) {
            throw new MatrixException("Размеры матриц не совпадают");
        }
        else {
            int[][] matr = new int[matrix.rows][matrix.columns];
            for (int i = 0; i < matrix.rows; i++) {
                for (int j = 0; j < matrix.columns; j++) {
                    matr[i][j] = this.matrix[i][j] + matrix.matrix[i][j];
                }
            }
            return new Matrix(matr, matrix.rows, matrix.rows);
        }
    }

    public final Matrix sub(Matrix matrix) throws MatrixException {
        if (matrix.rows != this.rows || matrix.columns != this.columns) {
            throw new MatrixException("Размеры матриц не совпадают");
        }
        else {
            int[][] result = new int[matrix.rows][matrix.columns];
            for (int i = 0; i < matrix.rows; i++) {
                for (int j = 0; j < matrix.columns; j++) {
                    result[i][j] = this.matrix[i][j] - matrix.matrix[i][j];
                }
            }
            return new Matrix(result, matrix.rows, matrix.rows);
        }
    }

    public final Matrix mul(Matrix matrix) throws MatrixException {
        if (this.columns != matrix.rows) {
```

```

        throw new MatrixException("Количество столбцов первой " +
            "матрицы не равно количеству строк второй матрицы");
    }
    else {
        int[][] result = new int[this.rows][matrix.columns];
        for (int i = 0; i < this.rows; i++) {
            for (int j = 0; j < matrix.columns; j++) {
                for (int k = 0; k < this.columns; k++) {
                    result[i][j] += this.matrix[i][k] * matrix.matrix[k][j];
                }
            }
        }
        return new Matrix(result, this.rows, matrix.columns);
    }
}

public final Boolean equal(Matrix matrix) throws MatrixException {
    if (matrix.rows != this.rows || matrix.columns != this.columns) {
        throw new MatrixException("Размеры матриц не совпадают");
    }
    else {
        boolean isEqual = true;
        for (int i = 0; i < matrix.rows && isEqual; i++) {
            for (int j = 0; j < matrix.columns && isEqual; j++) {
                if (this.matrix[i][j] != matrix.matrix[i][j]) {
                    isEqual = false;
                }
            }
        }
        return isEqual;
    }
}

public final Matrix transpose() throws MatrixException{
    if (this.columns != this.rows) {
        throw new MatrixException("Количество строк и столбцов должно совпадать");
    }
    int[][] result = new int[columns][rows];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            result[j][i] = this.matrix[i][j];
        }
    }
    return new Matrix(result, columns, rows);
}

public final int min() {
    int min = this.matrix[0][0];
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            if (this.matrix[i][j] < min) {
                min = this.matrix[i][j];
            }
        }
    }
}

```

```

    }
}
return min;
}

public final String convertToString() {
    StringBuilder result = new StringBuilder("{}");
    for (int i = 0; i < this.rows; i++) {
        result.append("{");
        for (int j = 0; j < this.columns; j++) {
            if (j + 1 < this.columns) {
                result.append(this.matrix[i][j]).append(",");
            }
            else {
                result.append(this.matrix[i][j]);
            }
        }
        result.append("}");
        if (i + 1 < this.rows) {
            result.append(",");
        }
    }
    result.append("}");
    return result.toString();
}

public final int getElement(int i, int j) throws MatrixException{
    if (i < 0 || i >= this.rows) {
        throw new MatrixException("Количество строк не входит в допустимый интервал");
    }
    if (j < 0 || j >= this.columns) {
        throw new MatrixException("Количество столбцов не входит в допустимый интервал");
    }
    else {
        return this.matrix[i][j];
    }
}

public final int getRows() {
    return this.rows;
}

public final int getColumns() {
    return this.columns;
}
}

```

```
package com.example.matrixapplication;

public class MatrixException extends Exception {

    private String cause;

    MatrixException(String message) {
        super(message);
    }
}
```

```
package com.example.matrixapplication;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertFalse;
import static org.junit.Assert.assertTrue;

import org.junit.Test;

public class MatrixC2Test {

    @Test
    public void testConstructor_ValidPath() throws MatrixException {
        int[][] data = {{1, 2}, {3, 4}};
        Matrix matrix = new Matrix(data, 2, 2);
        assertEquals(2, matrix.getRows());
        assertEquals(2, matrix.getColumns());
    }

    @Test(expected = MatrixException.class)
    public void testConstructor_InvalidRowsPath() throws MatrixException {
        int[][] data = {{1, 2}};
        new Matrix(data, 0, 2);
    }

    @Test(expected = MatrixException.class)
    public void testConstructor_InvalidColumnsPath() throws MatrixException {
        int[][] data = {{1, 2}};
        new Matrix(data, 1, -1);
    }

    @Test
    public void testSum_ValidPath() throws MatrixException {
        int[][] data1 = {{1, 2}, {3, 4}};
        int[][] data2 = {{5, 6}, {7, 8}};
        Matrix m1 = new Matrix(data1, 2, 2);
        Matrix m2 = new Matrix(data2, 2, 2);

        Matrix result = m1.sum(m2);
        assertEquals(6, result.getElement(0, 0));
        assertEquals(8, result.getElement(0, 1));
    }

    @Test(expected = MatrixException.class)
    public void testSum_InvalidDimensionsPath_RowsDifferent() throws MatrixException {
        int[][] data1 = {{1, 2}, {3, 4}};
        int[][] data2 = {{1, 2}};
        Matrix m1 = new Matrix(data1, 2, 2);
        Matrix m2 = new Matrix(data2, 1, 2);
        m1.sum(m2);
    }
}
```



```

@Test(expected = MatrixException.class)
public void testSum_InvalidDimensionsPath_ColumnsDifferent() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{1, 2, 3}, {4, 5, 6}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 3);
    m1.sum(m2);
}

```

```

@Test
public void testSub_ValidPath() throws MatrixException {
    int[][] data1 = {{5, 6}, {7, 8}};
    int[][] data2 = {{1, 2}, {3, 4}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 2);

    Matrix result = m1.sub(m2);
    assertEquals(4, result.getElement(0, 0));
    assertEquals(4, result.getElement(0, 1));
}

```

```

@Test(expected = MatrixException.class)
public void testSub_InvalidPathCols() throws MatrixException {
    int[][] data1 = {{5, 6}, {7, 8}};
    int[][] data2 = {{1, 2, 1}, {3, 4, 1}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 3);

    Matrix result = m1.sub(m2);
}

```

```

@Test(expected = MatrixException.class)
public void testSub_InvalidPathRows() throws MatrixException {
    int[][] data1 = {{5, 6}, {7, 8}};
    int[][] data2 = {{1, 2}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 1, 2);

    Matrix result = m1.sub(m2);
}

```

```

@Test
public void testMul_ValidPath() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{5, 6}, {7, 8}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 2);

    Matrix result = m1.mul(m2);
    assertEquals(19, result.getElement(0, 0)); // 1*5 + 2*7
    assertEquals(22, result.getElement(0, 1)); // 1*6 + 2*8
}

```

```
@Test(expected = MatrixException.class)
public void testMul_InvalidDimensionsPath() throws MatrixException {
    int[][] data1 = {{1, 2, 3}};
    int[][] data2 = {{1, 2}};
    Matrix m1 = new Matrix(data1, 1, 3);
    Matrix m2 = new Matrix(data2, 2, 2);
    m1.mul(m2);
}
```

```
@Test
public void testEqual_ValidPath_EqualMatrices() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix m1 = new Matrix(data, 2, 2);
    Matrix m2 = new Matrix(data, 2, 2);

    assertTrue(m1.equal(m2));
}
```

```
@Test
public void testEqual_ValidPath_NotEqualMatrices() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{1, 2}, {3, 5}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 2);

    assertFalse(m1.equal(m2));
}
```

```
@Test
public void testEqual_ValidPath_NotEqualFirstElement() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{0, 2}, {3, 4}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 2, 2);

    assertFalse(m1.equal(m2));
}
```

```
@Test(expected = MatrixException.class)
public void testEqual_InvalidPathRows() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{1, 2}};
    Matrix m1 = new Matrix(data1, 2, 2);
    Matrix m2 = new Matrix(data2, 1, 2);
    m1.equal(m2);
}
```

```
@Test(expected = MatrixException.class)
public void testEqual_InvalidPathCols() throws MatrixException {
    int[][] data1 = {{1, 2}, {3, 4}};
    int[][] data2 = {{1, 2, 3}, {4, 5, 6}};
```

```

Matrix m1 = new Matrix(data1, 2, 2);
Matrix m2 = new Matrix(data2, 2, 3);
m1.equal(m2);
}

@Test
public void testTranspose_ValidPath() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix original = new Matrix(data, 2, 2);

    Matrix transposed = original.transpose();
    assertEquals(1, transposed.getElement(0, 0));
    assertEquals(3, transposed.getElement(0, 1));
    assertEquals(2, transposed.getElement(1, 0));
    assertEquals(4, transposed.getElement(1, 1));
}

@Test(expected = MatrixException.class)
public void testTranspose_InvalidPath() throws MatrixException {
    int[][] data = {{1, 2, 3}, {4, 5, 6}};
    Matrix matrix = new Matrix(data, 2, 3);
    matrix.transpose();
}

@Test
public void testMin_Path_MinFirstElement() throws MatrixException {
    int[][] data = {{1, 2, 3}, {4, 5, 6}};
    Matrix matrix = new Matrix(data, 2, 3);
    assertEquals(1, matrix.min());
}

@Test
public void testMin_Path_MinInMiddle() throws MatrixException {
    int[][] data = {{5, 3, 8}, {2, 7, 1}, {4, 6, 9}};
    Matrix matrix = new Matrix(data, 3, 3);
    assertEquals(1, matrix.min());
}

@Test
public void testMin_Path_MinLastElement() throws MatrixException {
    int[][] data = {{5, 4, 3}, {2, 1, 0}};
    Matrix matrix = new Matrix(data, 2, 3);
    assertEquals(0, matrix.min());
}

@Test
public void testMin_Path_SingleElement() throws MatrixException {
    int[][] data = {{42}};
    Matrix matrix = new Matrix(data, 1, 1);
    assertEquals(42, matrix.min());
}

```

```

@Test
public void testConvertToString_Path_1x1() throws MatrixException {
    int[][] data = {{42}};
    Matrix matrix = new Matrix(data, 1, 1);
    assertEquals("{42}", matrix.convertToString());
}

@Test
public void testConvertToString_Path_1x2() throws MatrixException {
    int[][] data = {{1, 2}};
    Matrix matrix = new Matrix(data, 1, 2);
    assertEquals("{1,2}", matrix.convertToString());
}

@Test
public void testConvertToString_Path_2x1() throws MatrixException {
    int[][] data = {{1}, {2}};
    Matrix matrix = new Matrix(data, 2, 1);
    assertEquals("{1},{2}", matrix.convertToString());
}

@Test
public void testConvertToString_Path_2x2() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    assertEquals("{1,2},{3,4}", matrix.convertToString());
}

@Test
public void testConvertToString_Path_3x2() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}, {5, 6}};
    Matrix matrix = new Matrix(data, 3, 2);
    assertEquals("{1,2},{3,4},{5,6}", matrix.convertToString());
}

@Test
public void testGetElement_ValidPath() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    assertEquals(3, matrix.getElement(1, 0));
}

@Test(expected = MatrixException.class)
public void testGetElement_InvalidRowPath_Negative() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(-1, 0);
}

@Test(expected = MatrixException.class)

```

```

public void testGetElement_InvalidRowPath_TooLarge() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(5, 0);
}

@Test(expected = MatrixException.class)
public void testGetElement_InvalidColumnPath_Negative() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(0, -1);
}

@Test(expected = MatrixException.class)
public void testGetElement_InvalidColumnPath_TooLarge() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(0, 5);
}

@Test
public void testGetRows_Path() throws MatrixException {
    int[][] data = {{1, 2, 3}, {4, 5, 6}};
    Matrix matrix = new Matrix(data, 2, 3);
    assertEquals(2, matrix.getRows());
}

@Test
public void testGetColumns_Path() throws MatrixException {
    int[][] data = {{1, 2, 3}, {4, 5, 6}};
    Matrix matrix = new Matrix(data, 2, 3);
    assertEquals(3, matrix.getColumns());
}

@Test
public void testGetElement_BoundaryValues() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);

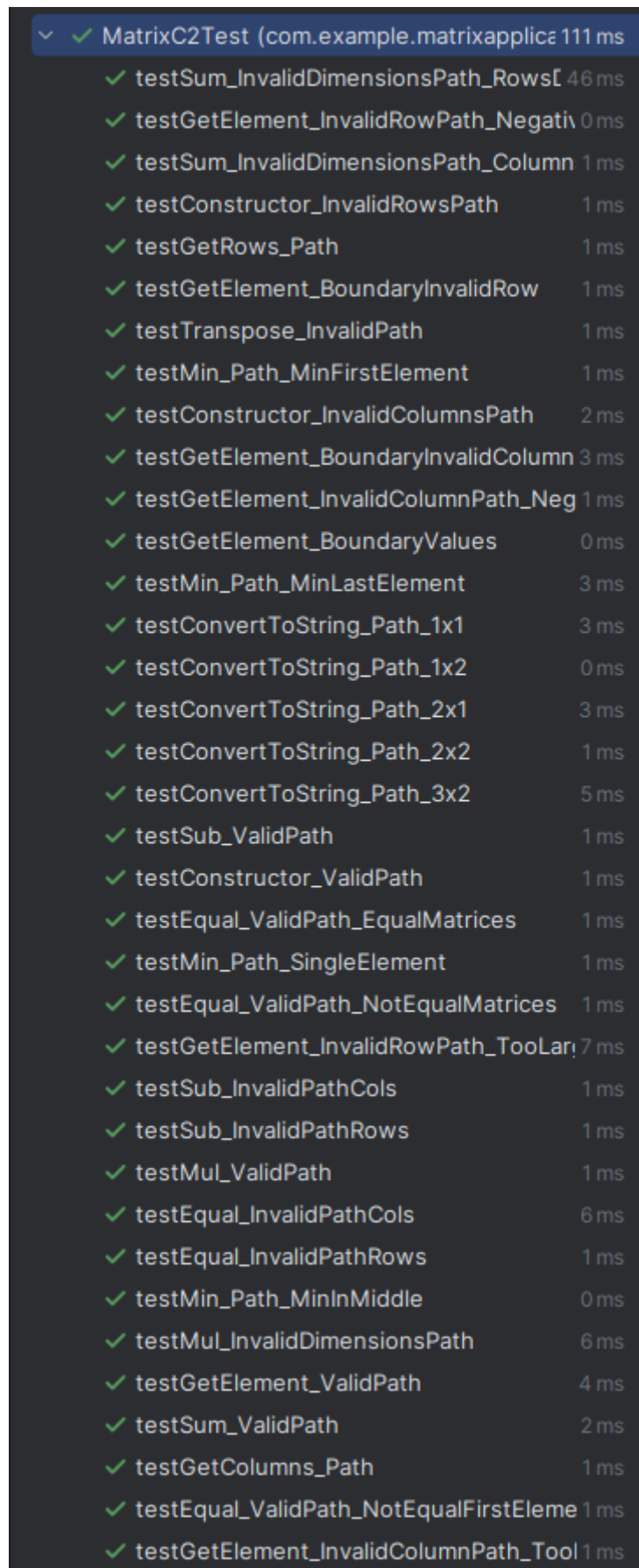
    assertEquals(1, matrix.getElement(0, 0));
    assertEquals(2, matrix.getElement(0, 1));
    assertEquals(3, matrix.getElement(1, 0));
    assertEquals(4, matrix.getElement(1, 1));
}

@Test(expected = MatrixException.class)
public void testGetElement_BoundaryInvalidRow() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(2, 0);
}

```

```
@Test(expected = MatrixException.class)
public void testGetElement_BoundaryInvalidColumn() throws MatrixException {
    int[][] data = {{1, 2}, {3, 4}};
    Matrix matrix = new Matrix(data, 2, 2);
    matrix.getElement(0, 2);
}
}
```

Результаты выполнения модульных тестов

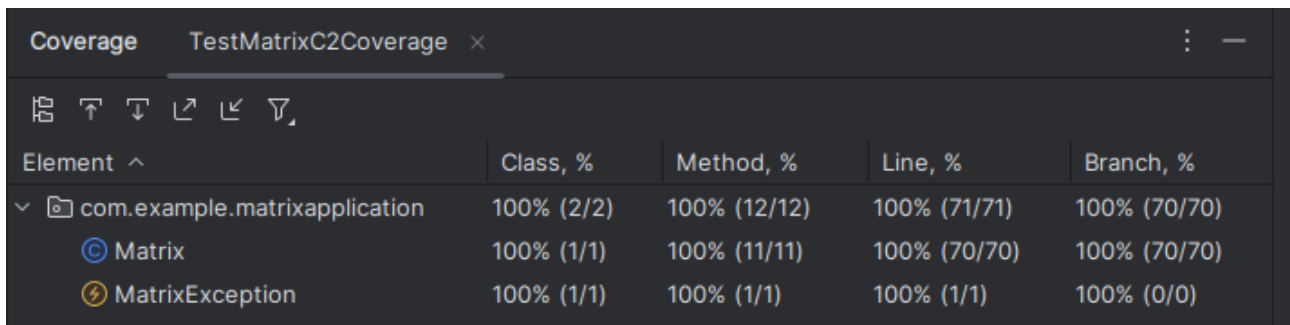


The image shows a screenshot of a test runner interface, likely IntelliJ IDEA, displaying the results of a JUnit test suite for the class `MatrixC2Test`. The root entry is `MatrixC2Test (com.example.matrixapplicat 111 ms)`, which is expanded to show 40 individual test cases. Each test case is marked with a green checkmark, indicating it passed. The test names and their execution times in milliseconds are listed below.

✓ testSum_InvalidDimensionsPath_Rows	46 ms
✓ testGetElement_InvalidRowPath_Negative	0 ms
✓ testSum_InvalidDimensionsPath_Column	1 ms
✓ testConstructor_InvalidRowsPath	1 ms
✓ testGetRows_Path	1 ms
✓ testGetElement_BoundaryInvalidRow	1 ms
✓ testTranspose_InvalidPath	1 ms
✓ testMin_Path_MinFirstElement	1 ms
✓ testConstructor_InvalidColumnsPath	2 ms
✓ testGetElement_BoundaryInvalidColumn	3 ms
✓ testGetElement_InvalidColumnPath_Negative	1 ms
✓ testGetElement_BoundaryValues	0 ms
✓ testMin_Path_MinLastElement	3 ms
✓ testConvertToString_Path_1x1	3 ms
✓ testConvertToString_Path_1x2	0 ms
✓ testConvertToString_Path_2x1	3 ms
✓ testConvertToString_Path_2x2	1 ms
✓ testConvertToString_Path_3x2	5 ms
✓ testSub_ValidPath	1 ms
✓ testConstructor_ValidPath	1 ms
✓ testEqual_ValidPath_EqualMatrices	1 ms
✓ testMin_Path_SingleElement	1 ms
✓ testEqual_ValidPath_NotEqualMatrices	1 ms
✓ testGetElement_InvalidRowPath_TooLarge	7 ms
✓ testSub_InvalidPathCols	1 ms
✓ testSub_InvalidPathRows	1 ms
✓ testMul_ValidPath	1 ms
✓ testEqual_InvalidPathCols	6 ms
✓ testEqual_InvalidPathRows	1 ms
✓ testMin_Path_MinInMiddle	0 ms
✓ testMul_InvalidDimensionsPath	6 ms
✓ testGetElement_ValidPath	4 ms
✓ testSum_ValidPath	2 ms
✓ testGetColumns_Path	1 ms
✓ testEqual_ValidPath_NotEqualFirstElement	1 ms
✓ testGetElement_InvalidColumnPath_Tool	1 ms

Рис. 1 — Результаты тестов для класса Matrix.

Результаты покрытия разработанного кода тестами



The screenshot shows the 'Coverage' tool window in IntelliJ IDEA. The title bar reads 'Coverage TestMatrixC2Coverage'. Below the title bar is a toolbar with icons for various coverage actions. The main area is a table with the following columns: 'Element', 'Class, %', 'Method, %', 'Line, %', and 'Branch, %'. The table contains three rows of data, all showing 100% coverage.

Element ^	Class, %	Method, %	Line, %	Branch, %
com.example.matrixapplication	100% (2/2)	100% (12/12)	100% (71/71)	100% (70/70)
Matrix	100% (1/1)	100% (11/11)	100% (70/70)	100% (70/70)
MatrixException	100% (1/1)	100% (1/1)	100% (1/1)	100% (0/0)

Рис. 2 — Результат покрытия кода тестами с помощью средств IntelliJ IDEA.

Выводы по проделанной работе

В результате выполнения практической работы был разработан класс, методы которого соответствуют предоставленной к практической работе спецификации. Был разработан набор модульных тестов для проверки корректности работы методов разработанного класса. В ходе выполнения работы были получены навыки разработки модульных тестов для тестирования методов классов и выполнения модульного тестирования на языке Java с помощью средств автоматизации IntelliJ IDEA.